

WEEK-2 PRACTICAL

Task 1: Develop a C program that

demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    char command[100];
    pid_t pid;

    printf("Enter a Linux command: ");
    scanf("%s", command);

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed.\n");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process ID: %d\n", getpid());
        printf("Parent Process ID: %d\n", getppid());

        execlp(command, command, (char *)NULL);

        printf("Command execution failed.\n");
        exit(1);
    }
    else
    {
        printf("Parent Process ID: %d\n", getpid());

        wait(NULL);
    }
}
```

Child process completed.

OUTPUT:

```
lenovo@Manya:~/Week2$ nano task1.c
lenovo@Manya:~/Week2$ gcc task1.c -o task1
lenovo@Manya:~/Week2$ ls
tas1 task1 task1.c
lenovo@Manya:~/Week2$ ./task1
Enter a Linux command: uname
Parent Process ID: 852
Child Process ID: 860
Parent Process ID: 852
Linux
Child process completed.
lenovo@Manya:~/Week2$ |
```

Child process completed.

REPORT: I executed the C program successfully. The program accepted a Linux command from the user, created a child process using for The `uname -a` command displayed complete information about my Linux system, including the kernel version, system architecture, hostname, and operating system details.`k()`, and executed the command using the `exec()` system call. The parent process waited until the child process finished execution. Finally, the program displayed the process IDs of both the parent and child processes, and the message "Child process completed." was displayed.

Task-2: Using Linux terminal commands (`uname`, `lscpu`, `ps`, `top`), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Uname-a:

```
lenovo@Manya:~/Week2$ uname -a
Linux Manya 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 GNU/Linux
lenovo@Manya:~/Week2$ |
```

REPORT: The `uname -a` command displayed complete information about my Linux system, including the kernel version, system architecture, hostname, and operating system details.

Lscpu:

```

lenovo@Manya:~/Week2$ lscpu
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:                39 bits physical, 48 bits virtual
Byte Order:                   Little Endian
CPU(s):                       4
On-line CPU(s) list:         0-3
Vendor ID:                    GenuineIntel
Model name:                   Intel(R) Core(TM) i3-1005G1 CPU @ 1.20GHz
CPU family:                   6
Model:                        126
Thread(s) per core:          2
Core(s) per socket:          2
Socket(s):                    1
Stepping:                     5
BogoMIPS:                     2380.80
Flags:                        fpu vme de pse tsc msr pae mce cx8 apic sep mtr
                             r pge mca cmov pat pse36 clflush mmx fxsr sse s
                             se2 ss ht syscall nx pdpe1gb rdtscp lm constant
                             _tsc arch_perfmon rep_good noopl xtopology tsc_r
                             eliable nonstop_tsc cpuid tsc_known_freq pni pc
                             lmulqdq vmx ssse3 fma cx16 pdcm pcid sse4_1 sse
                             4_2 x2apic movbe popcnt tsc_deadline_timer aes
                             xsave avx f16c rdrand hypervisor lahf_lm abm 3d
                             nowprefetch ssbd ibrs ibpb stibp ibrs_enhanced
                             tpr_shadow ept vpid ept_ad fsgsbase tsc_adjust
                             bmi1 avx2 smep bmi2 erms invpcid avx512f avx512
                             dq rdseed adx smap avx512ifma clflushopt avx512
                             cd sha_ni avx512bw avx512vl xsaveopt xsavec xge
                             tbnl xsaves vnmi avx512vbmi umip avx512_vbmi2 g
                             fni vaes vpcvmulqdq avx512_vnni avx512_bitalg a
                             vx512_vpocntdq rdpid fsrm md_clear flush_lld a
                             rch_capabilities

Virtualization features:
Virtualization:               VT-x
Hypervisor vendor:            Microsoft
Virtualization type:          full
Caches (sum of all):
L1d:                           96 KiB (2 instances)
L1i:                           64 KiB (2 instances)
L2:                             1 MiB (2 instances)
L3:                             4 MiB (1 instance)
NUMA:
NUMA node(s):                  1
NUMA node0 CPU(s):            0-3
Vulnerabilities:
Gather data sampling:          Unknown: Dependent on hypervisor status
Ghostwrite:                    Not affected
Indirect target selection:      Mitigation; Aligned branch/return thunks
Itlb multihit:                 KVM: Mitigation: Split huge pages
L1tf:                           Not affected
Mds:                            Not affected
Meltdown:                      Not affected
Mmio stale data:               Vulnerable: Clear CPU buffers attempted, no mic
                                rocode; SMT Host state unknown
Old microcode:                 Not affected
Reg file data sampling:         Not affected
Retbleed:                      Mitigation; Enhanced IBRS
Spec rstack overflow:          Not affected
Spec store bypass:              Mitigation; Speculative Store Bypass disabled v
                                ia prctl
Spectre v1:                    Mitigation; usercopy/swaps barriers and __user
                                pointer sanitization
Spectre v2:                    Mitigation; Enhanced / Automatic IBRS; IBPB con
                                ditional; PBRSE-eIBRS SW sequence; BHI SW loop,
                                KVM SW loop
Srbds:                         Unknown: Dependent on hypervisor status
Tsa:                           Not affected
Tsx async abort:               Not affected
Vmscape:                       Not affected
lenovo@Manya:~/Week2$ |

```

REPORT: The lscpu command displayed the CPU information of my system, such as the processor architecture, number of CPU cores, CPU model, and other hardware details. This helped me understand the processor resources available.

Ps:

```

lenovo@Manya:~/Week2$ ps
  PID TTY          TIME CMD
   331 pts/0        00:00:00 bash
  1024 pts/0        00:00:00 ps
lenovo@Manya:~/Week2$ |

```

REPORT: The ps command displayed the processes currently running in my terminal. I observed the process IDs and command names of the active processes.

Top:

```
lenovo@lenovo:~/week2$ top
top - 09:15:52 up 23 min, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 26 total, 1 running, 25 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3812.6 total, 3320.5 free, 488.2 used, 142.1 buff/cache
MiB Swap: 1824.0 total, 1824.0 free, 0.0 used, 3324.4 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1834	lenovo	20	0	9196	7140	3964	R	100.0	0.2	0:00.04	top
1	root	20	0	24212	15396	11596	S	0.0	0.4	0:02.00	systemd
2	root	20	0	3180	2212	2072	S	0.0	0.1	0:00.11	init-systemd(Ub
6	root	20	0	3180	2048	1940	S	0.0	0.1	0:00.00	init
55	root	19	-1	50400	16768	15496	S	0.0	0.4	0:00.51	systemd-journal
87	systemd+	20	0	22416	14568	12068	S	0.0	0.4	0:00.11	systemd-resolve
96	root	20	0	35216	12272	9340	S	0.0	0.3	0:01.27	systemd-udev
116	root	20	0	2888	2012	1884	S	0.0	0.1	0:00.06	chronyd-starter
117	root	20	0	4472	3236	2988	S	0.0	0.1	0:00.01	cron
118	message+	20	0	8800	5576	4892	S	0.0	0.1	0:00.09	dbus-daemon
122	root	20	0	44096	29864	14728	S	0.0	0.8	0:00.36	networkd-dispat
150	root	20	0	18440	9484	8240	S	0.0	0.2	0:00.13	systemd-logind
179	syslog	20	0	228548	5996	4712	S	0.0	0.2	0:00.13	rsyslogd
216	_chrony	20	0	23896	11516	7308	S	0.0	0.3	0:00.16	chronyd
220	root	20	0	123460	32592	17984	S	0.0	0.0	0:00.32	unattended-upgr
231	root	20	0	5492	2012	2628	S	0.0	0.1	0:00.01	agetty
232	_chrony	20	0	12096	2496	1852	S	0.0	0.1	0:00.01	chronyd
324	root	20	0	3184	1116	980	S	0.0	0.0	0:00.00	SessionLeader
326	root	20	0	3200	1252	1108	S	0.0	0.0	0:00.12	Relay(331)
331	lenovo	20	0	6268	5584	3912	S	0.0	0.1	0:00.28	bash
333	root	20	0	8644	5612	4772	S	0.0	0.1	0:00.14	login
380	lenovo	20	0	22364	13640	11028	S	0.0	0.3	0:00.57	systemd
382	lenovo	20	0	22912	4112	2112	S	0.0	0.1	0:00.00	(sd-pam)
422	lenovo	20	0	6136	5472	3928	S	0.0	0.1	0:00.06	bash
1835	root	20	0	35220	6356	3392	S	0.0	0.2	0:00.00	(udev-worker)
1836	root	20	0	35220	6356	3392	S	0.0	0.2	0:00.00	(udev-worker)

REPORT: The top command displayed the live status of the system. I observed the CPU usage, memory usage, running processes, and system performance updating continuously.